RemixBuddy

Concept & Technical Decision Document

Team-ready draft for implementation planning

Document purpose	**Engineering concept and decision record, not a pitch deck**
Recommended product shape	Hybrid local-first system: standalone processing engine + VST3 plugin UI/client
MVP focus	Track load, BPM/key analysis, stem extraction, remix-pack export
Processing mode	Offline / asynchronous processing; no hard realtime stem separation in MVP
Deployment	Local only in MVP; cloud out of scope

1. Executive summary

RemixBuddy exists to compress remix preparation from a fragmented multi-tool workflow into one controlled local workflow. The concept is deliberately narrowed for implementation: the MVP is not an AI composer and not a mastering suite. It is a remix-preparation product that loads an audio file, analyzes BPM/key, extracts stems, and exports a remix pack.

- Primary user: electronic music producer working in genres such as trance, house, techno, or EDM.
- Primary value: reduce setup time before creative work starts.
- MVP recommendation: local-first hybrid system with a standalone processing engine and a thin VST3 client/UI.
- Reason for hybrid recommendation: long-running DSP and ML jobs should not block or destabilize a DAW host.
- Out of scope for MVP: cloud processing, AAX/AU, live stem separation, full arrangement generation.

2. Problem definition

2.1 Why the project exists

Current remix preparation is fragmented. A producer commonly needs one tool for stem extraction, another for BPM or key validation, manual listening for structure discovery, and manual export steps to build reusable loops. This wastes time, creates repeated hand-work, and introduces avoidable friction before creative production even starts.

2.2 Target audience

Audience	Description	Priority
Electronic music producers	Users creating remixes or bootlegs inside a DAW; strongest fit for trance, house, techno, EDM.	Primary
DJs / mashup creators	Users preparing hooks, acapellas, and edit material.	Secondary
Content creators / sample designers	Users extracting usable loop material from songs or references.	Secondary

2.3 Use cases

Use case	User goal	Product outcome
Remix creation	Turn an existing track into production-ready source material.	Stems + BPM/key/structure metadata + loops.
Vocal sampling	Find and export usable vocal hooks quickly.	Vocal stem + hook candidates + export markers.
Track analysis	Understand a track before building a remix.	Detected BPM, key, sections, and confidence.

2.4 Concrete scenarios

- A producer drops a 100 BPM pop song into RemixBuddy and exports a 136 BPM trance-ready vocal pack.
- A producer checks whether a downloaded acapella is in the same key as the planned remix project.
- A producer wants to identify the best 8-bar hook section without manual scrubbing through the full song.

3. Product decisions and non-functional requirements

3.1 Product shape

- Decision: MVP is local-first and hybrid.
- Standalone processing service handles decoding, normalization, analysis, and stem extraction.
- VST3 plugin acts as client/UI inside the DAW and communicates with the local service asynchronously.
- Standalone-only mode should exist for users who want batch preparation outside the DAW.

3.2 Supported platforms

- MVP target: Windows 10/11 x64.
- macOS can be evaluated later but is not required for first implementation planning unless the team explicitly chooses cross-platform from day one.
- Linux is out of scope for MVP.

3.3 Performance / latency targets

Category	Target / decision
UI responsiveness	No blocking UI actions longer than 200 ms; long jobs run async.
Track analysis latency	< 20 seconds for a 5-minute stereo track on recommended hardware.
Stem extraction latency	< 60 seconds for a 5-minute stereo track on recommended GPU hardware; CPU fallback allowed to be slower.
Export latency	< 10 seconds after processing is complete.
Realtime processing	Not required in MVP. Stem separation is offline / render-based.

3.4 Hardware requirements

Tier	CPU	RAM	GPU	Expected experience
Minimum	4 cores / 8 threads	16 GB	No GPU required	Analysis works; stem extraction slower.
Recommended	8 cores / 16 threads	32 GB	NVIDIA GPU with >= 8 GB VRAM	Comfortable processing times for MVP.
Unsupported	< 16 GB RAM	< 4 cores	Integrated-only low-end systems if memory constrained	May fail or be too slow for practical use.

3.5 File and format constraints

Requirement	Decision
Supported input formats (MVP)	WAV, MP3, FLAC
Channel support	Mono and stereo; multichannel out of scope in MVP
Max file length (supported)	Up to 30 minutes in MVP; >30 minutes explicitly unsupported for first release
Recommended file length	Up to 10 minutes for best user experience
Working audio representation	32-bit float PCM
Internal sample rates	Normalize to 44.1 kHz or 48 kHz working rate

3.6 Stability, logging, and monitoring

- The processing service must survive bad inputs without crashing the host DAW.
- Plugin and service logs are separate. The plugin logs UI events and job requests; the service logs DSP/model events and failures.
- Structured local logging format: JSONL per session plus human-readable text log.
- Each job gets a job_id for traceability across plugin UI and processing service.
- Crash dumps and telemetry to cloud are out of scope for MVP unless explicitly enabled later.

4. System architecture

The system is divided into a host-facing client and a local processing backend. This is a deliberate architectural decision: heavy DSP/ML workloads should not live inside a fragile DAW callback path if the product needs to remain stable.

4.1 High-level data flow

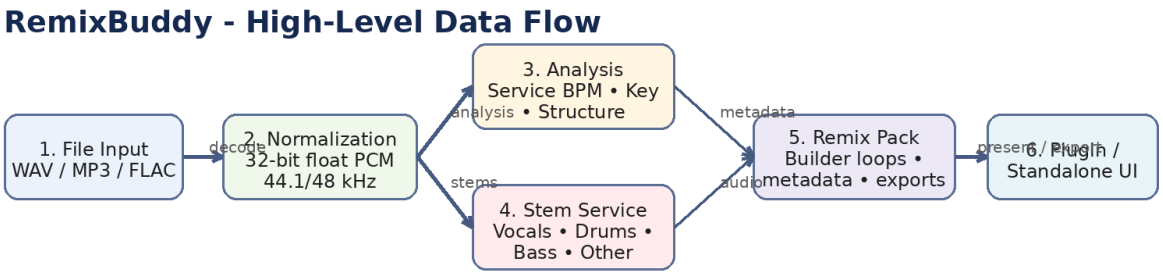

Figure 1. High-level data flow from input decoding to remix-pack output.

4.2 Module overview

Module	Responsibility	Proposed technology / notes
File Input / Decoder	Load, decode, validate, normalize metadata.	FFmpeg/libav or equivalent decoding layer.
Audio Analysis Engine	BPM, key, confidence, basic structure hints.	Essentia/librosa or equivalent; decision by implementation team.
Stem Separation Engine	Produce vocals, drums, bass, other (minimum).	Demucs-family or chosen replacement model; local inference only in MVP.
Tempo Conversion Engine	Convert output stems or packs to target BPM.	Rubber Band or equivalent high-quality offline stretcher.
Structure Detection Engine	Generate section candidates and loop regions.	Heuristic + MIR-based segmentation; confidence required.
Remix Pack Builder	Assemble exports and metadata.	Local file writer + JSON manifest.
Plugin / Standalone UI	User interaction, job status, preview of results.	JUCE VST3 UI or equivalent native UI framework.

4.3 Module communication

Modules communicate through explicit job contracts. The plugin sends requests to the processing service, the service writes status updates, and the UI polls or subscribes to progress. Internal audio buffers are not passed through the plugin for heavy processing once a job starts.

Interface	Direction	Payload type
Plugin -> Service	Request	JSON command with file path, settings, and job_id
Service -> Plugin	Status	JSON progress / result message
Service -> Disk	Output	WAV/FLAC audio + JSON manifest + optional MIDI

5. Interfaces and contracts

5.1 Inputs

Input	Type	Notes
Audio file	WAV / MP3 / FLAC	User-supplied local file path
Target BPM	Float	Optional for analysis; required for tempo conversion export
Stem selection	Array	Example: ['vocals','drums','bass','other']
Quality preset	Enum	draft | standard | high
Output path	String	Writable local directory

5.2 Outputs

Output	Format	Definition
Stem audio	WAV 24-bit	Per-stem export; FLAC optional later
Remix pack manifest	JSON	Metadata, timings, key, BPM, confidence, file locations
Loop exports	WAV	Optional region-based clips
Chord / marker export	MIDI or JSON markers	Optional in post-MVP if harmonic engine validated

5.3 Example JSON request

```
{
  "job_id": "rb-2026-00125",
  "command": "analyze_and_extract",
  "input_file": "D:/Audio/source_track.wav",
```

```
  "working_sample_rate": 48000,
  "quality_preset": "standard",
  "stems": ["vocals", "drums", "bass", "other"],
  "target_bpm": 136.0,
  "output_dir": "D:/Exports/RemixBuddy/track_001"
}
```


5.4 Example JSON result


```
{
  "job_id": "rb-2026-00125",
  "status": "completed",
  "analysis": {
    "bpm": 100.04,
    "bpm_confidence": 0.97,
    "key": "F minor",
    "key_confidence":  0.91
  },
  "sections": [
    {"label": "verse", "start_sec": 32.4, "end_sec": 61.8, "confidence": 0.73},
    {"label": "hook", "start_sec": 61.8, "end_sec": 89.2, "confidence": 0.82}
  ],
  "outputs": {
    "vocals": "vocals_136.wav",
    "drums": "drums.wav",
    "manifest": "remix_pack.json"
  }
}
```


6. User stories and acceptance criteria

US-01 Load track

As a producer, I want to load an audio file so that I can analyze BPM and key.

- Supported formats: WAV, MP3, FLAC
- Unsupported file returns a visible error without crashing
- File load result visible in UI within 2 seconds
- Track duration and filename shown after load

US-02 Analyze track

As a producer, I want BPM and key analysis so that I know how to prepare the remix.

- Analysis time < 20 seconds for a 5-minute stereo file on recommended hardware
- BPM confidence and key confidence shown in UI
- If confidence is below threshold, result is flagged 'review needed'
- No DAW host freeze during analysis

US-03 Extract stems

As a producer, I want vocal and instrument stems so that I can start remix production immediately.

- Stem export supports vocals, drums, bass, other at minimum
- Export format: WAV 24-bit
- Processing runs asynchronously
- Results include file paths and job completion status

US-04 Export remix pack

As a producer, I want one export action that creates a usable remix folder so that I do not manually assemble assets.

- Export folder contains stems plus JSON manifest
- Manifest records BPM, key, confidence, and source filename
- Export operation finishes without overwriting files unless explicitly allowed

7. UX flows, states, and error handling

7.1 Primary user journey

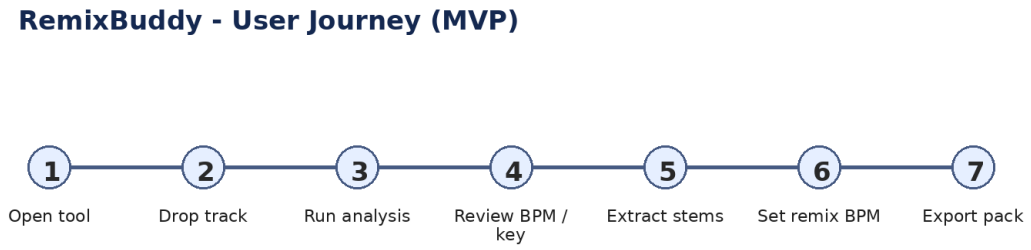

Figure 2. Primary MVP user journey.

7.2 Processing state model

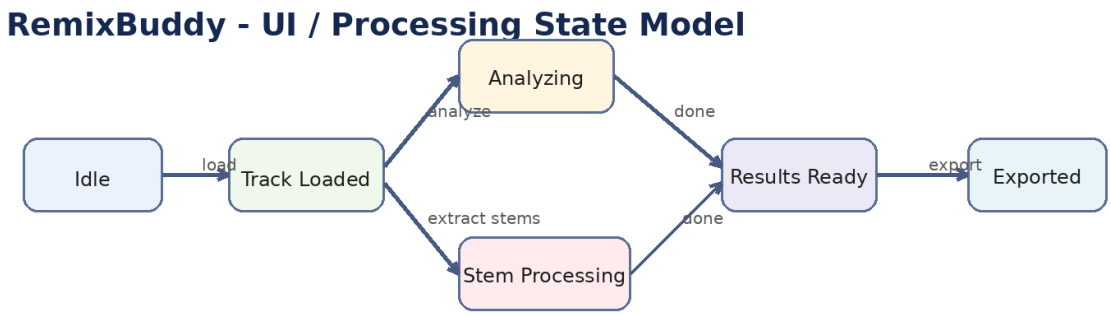

Figure 3. UI and processing states.

7.3 Plugin interaction model

- Idle state: no track loaded; only file input actions enabled.
- Track loaded state: analysis actions enabled; extraction/export disabled until analysis completes or user explicitly skips.
- Analyzing / processing states: progress visible; cancel action available; non-related controls disabled.
- Results ready state: export and audition/preview actions enabled.
- Error state: user sees actionable message and log reference; system returns safely to previous stable state.

7.4 GUI mockup placement

Figure 4. Current internal GUI mockup for discussion only.

7.5 Error handling examples

Situation	Expected behavior
Unsupported codec	Show validation error; do not create job
30+ minute file	Warn unsupported for MVP; user can cancel or continue at own risk only if team later enables it
Low-confidence BPM/key	Return result with visible 'review needed' badge
GPU unavailable	Fallback to CPU mode if supported; warn about slower processing
Write path invalid	Block export and show corrective message

8. Risks, limitations, and unsupported cases

8.1 Technical risks

- Stem quality is source-dependent; dense mixes, heavy sidechain pumping, or reverberant vocals can degrade separation.
- Large tempo jumps can create audible artifacts even with high-quality offline stretching.
- Structure detection is probabilistic; not every song exposes clear verse / chorus boundaries.
- Host integration risk exists if too much processing is attempted inside the plugin process.

8.2 Edge cases that must be explicitly defined

Case	MVP position
3-hour live set	Unsupported
Poor-quality / noisy source	Supported on best-effort basis; lower confidence expected
Tracks with no clear structure	Supported; may return 'structure unavailable'
BPM drift / live tempo fluctuation	Best-effort; no guarantee of single stable BPM
Multilingual vocals	Supported for separation; language understanding not required in MVP
Mono files	Supported
Multi-channel files	Unsupported

8.3 External dependencies and licensing topics

- The implementation team must select final libraries and verify commercial/license suitability before build start.
- Audio decoding, time-stretching, and ML inference dependencies must be reviewed for redistribution constraints.
- DAW support in MVP should be explicitly limited to VST3 unless the team decides otherwise after scoping.
- Cloud models and SaaS APIs are excluded from MVP to avoid legal, privacy, latency, and cost complexity.

9. MVP definition and roadmap

9.1 MVP definition

Included in MVP	Out of scope for MVP
File load (WAV/MP3/FLAC)	Cloud processing
BPM + key analysis	AAX / AU support
Stem extraction (vocals, drums, bass, other)	Realtime stem separation
Target BPM conversion for export pack	Automatic arrangement generation
Remix-pack export with JSON manifest	Advanced genre intelligence

9.2 Why this MVP

This is the smallest version that delivers real value: it removes the setup burden before creative remix work starts. It is also technically coherent because all included features can run offline and return deterministic artifacts.

9.3 Prioritized roadmap

Phase	Priority	Deliverables	Why

Phase 1 - Core MVP	Highest	Load, analyze, extract stems, export pack	Creates immediate producer value and validates the product thesis.
Phase 2 - Producer utilities	High	Structure detection, hook suggestions, target BPM presets, improved export options	Improves daily usability once the core pipeline is stable.
Phase 3 - Advanced intelligence	Medium	Genre-aware suggestions, vocal chop helpers, more detailed segmentation	Only worth adding after the core workflow is trusted.
Phase 4 - Platform expansion	Lower	Additional DAW formats and OS support	Should follow product-market validation, not precede it.

10. Definition of done and test strategy

10.1 Definition of done by module

Module	Definition of done
File load	Supported formats open reliably; invalid files rejected safely; metadata visible in UI.
Analysis	BPM/key returned with confidence; analysis completes within target time budget on recommended hardware.
Stem separation	All requested stems exported; service remains stable; export format is WAV 24-bit.
Export pack	Manifest written, files named consistently, no silent failures.

10.2 Test strategy

- Unit tests for file validation, request parsing, and export manifest generation.
- Benchmark tests for analysis and stem processing against a fixed internal test corpus.
- Regression tests to confirm repeated runs do not break prior outputs or host stability.
- Audio quality evaluation using reference tracks plus human listening review for MVP sign-off.
- Host integration smoke tests inside at least one supported VST3 DAW environment.

10.3 Suggested benchmark corpus

- Stereo pop track with clear vocals
- Dense EDM / trance mix
- Mono vocal-focused file
- Poor-quality MP3 source
- Track with BPM drift / live feel
- Track with ambiguous structure

11. Open decisions for the implementation team

- Final choice of decoding library
- Final model choice for local stem separation
- Final framework for service communication (embedded local server, IPC, or in-process worker manager)
- Whether MIDI/chord export belongs in MVP or Phase 2
- Whether macOS is required from day one

This document intentionally converts the initial RemixBuddy idea into an implementation-oriented concept. It records the product shape, scope boundaries, technical assumptions, contracts, and acceptance criteria needed for estimation and planning.